Day 1 · ช่วงบ่าย

ลงมือปฏิบัติด้วย Keras

เขียน neural network, CNN และ transfer learning ด้วยตนเอง

- MLP
- CNN
- Transfer Learning

แผนของช่วงบ่าย

ก่อนอื่น: เปิด Colab แล้วตั้ง Runtime เป็น GPU (Runtime > Change runtime type)

ส่วนที่ 1 — เขียน MLP ตัวแรก และเข้าใจ workflow การฝึกโมเดล

ส่วนที่ 2 — เปลี่ยนเป็น CNN บนข้อมูลเดียวกัน แล้วเทียบผลกับ MLP

ส่วนที่ 3 — Transfer learning บนรูปอาหาร ใช้โมเดลสำเร็จรูป

ส่วนที่ 4 — โปรเจค: เลือกดาต้าเซ็ตใหม่ แล้วลองหลายโมเดลด้วยตนเอง

ส่วนที่ 1

MLP บน Fashion-MNIST

neural network ตัวแรก และ workflow การฝึกโมเดล

~50 นาที



ส่วนที่ 1 · โหลดข้อมูล

โหลดชุดข้อมูลที่มากับ Keras

```
(x_train, y_train), \
(x_test, y_test) = \
  keras.datasets. \
  fashion_mnist. \
  load_data()

print(x_train.shape)
# (60000, 28, 28)
print(x_train.dtype)
# uint8  (0-255)
```

load_data() ดาวน์โหลดครั้งเดียว แล้วเก็บไว้ในเครื่อง

ชุดฝึก 60,000 รูป ชุดทดสอบ 10,000 รูป

x คือภาพ y คือหมายเลขประเภท 0-9

เช็ก .shape และ .dtype ทุกครั้งเป็นนิสัยที่ดี

ส่วนที่ 1 · สำรวจข้อมูล

ดูภาพจริงก่อนเสมอ

```
class_names = [
  'T-shirt','Trouser',
  'Pullover','Dress',
  'Coat','Sandal',
  'Shirt','Sneaker',
  'Bag','Ankle boot']

plt.imshow(x_train[0],
  cmap='gray')
plt.title(class_names[
  y_train[0]])
```

- ป้าย y เป็นเลข 0-9 จับคู่กับชื่อให้อ่านเข้าใจ
- imshow แสดงภาพ cmap='gray' เพราะเป็นภาพขาวดำ
- ยืนยันว่าข้อมูลกับป้ายตรงกันก่อนฝึก
- ขั้นที่คนมักข้าม แล้วเจอปัญหาทีหลัง

ส่วนที่ 1 · เตรียมข้อมูล

Normalization: ปรับค่าพิกเซลเป็น 0–1

```
# ก่อนปรับ: 0 ถึง 255
x_train = x_train / 255.0
x_test  = x_test  / 255.0
# หลังปรับ: 0.0 ถึง 1.0

print(x_train.min(),
      x_train.max())
# 0.0 1.0
```

- ค่าพิกเซลเดิม 0 (ดำ) ถึง 255 (ขาว)
- หารด้วย 255 ให้อยู่ในช่วง 0 ถึง 1
- ค่าที่ใหญ่ทำให้ gradient แกว่ง ฝึกไม่นิ่ง
- ต้องทำกับทั้ง train และ test ด้วยวิธีเดียวกัน

ส่วนที่ 1 · สร้างโมเดล

ขั้นที่ 1 — กำหนดโครงสร้าง MLP

```
mlp = keras.Sequential([
    layers.Input((28, 28)),
    layers.Flatten(),
    layers.Dense(128,
        activation='relu'),
    layers.Dense(10,
        activation='softmax')
])
```

- Sequential วางชั้นต่อกันเป็นลำดับ
- Flatten คลี่ภาพ 28×28 เป็นเวกเตอร์ 784 ค่า
- Dense(128, relu) คือ hidden layer จากตอนเช้า
- Dense(10, softmax) แจกความน่าจะเป็น 10 คลาส

ส่วนที่ 1 · workflow

สามขั้นตอนการสร้างโมเดลด้วย Keras

1 สร้างโมเดล

วางว่ามีชั้นอะไรบ้าง

→

2 compile

ตั้ง loss / optimizer

→

3 fit

สั่งฝึกด้วยข้อมูล

ทุกแล็บของช่วงบ่ายวนอยู่กับสามขั้นนี้ เปลี่ยนแค่ส่วนสร้างโครงสร้างในขั้นที่ 1

ส่วนที่ 1 · summary

อ่าน model.summary()

```
mlp.summary()

# Flatten -> (None, 784)
# Dense_1 -> (None, 128)
#   params = 100,480
# Dense_2 -> (None, 10)
#   params = 1,290
# Total   = 101,770
```

None คือมิติ batch ยังไม่กำหนด เติมตอนฝึก

Dense_1 = (784 × 128) + 128 = 100,480

params คือจำนวน weight ที่ต้องเรียนรู้

จำเลข 101,770 ไว้ เดี๋ยวเทียบกับ CNN

ส่วนที่ 1 · compile

ขั้นที่ 2 — ตั้งค่าการฝึก

```
mlp.compile(
  optimizer='adam',
  loss='sparse_
    categorical
    _crossentropy',
  metrics=['accuracy']
)
```

optimizer='adam' ใช้ค่าเริ่มต้นที่ดีอยู่แล้ว

loss คือสิ่งที่โมเดลพยายามทำให้น้อยลง

ใช้ sparse_ เพราะป้ายเป็นเลขจำนวนเต็ม ไม่ใช่ one-hot

metrics เป็นค่าที่เราดู แต่ไม่ใช้ปรับ weight

ส่วนที่ 1 · fit

ขั้นที่ 3 — สั่งฝึก

```
history = mlp.fit(
  x_train, y_train,
  epochs=10,
  batch_size=32,
  validation_split=0.1
)
```

fit เริ่มการฝึกจริง วนปรับ weight ตามข้อมูล

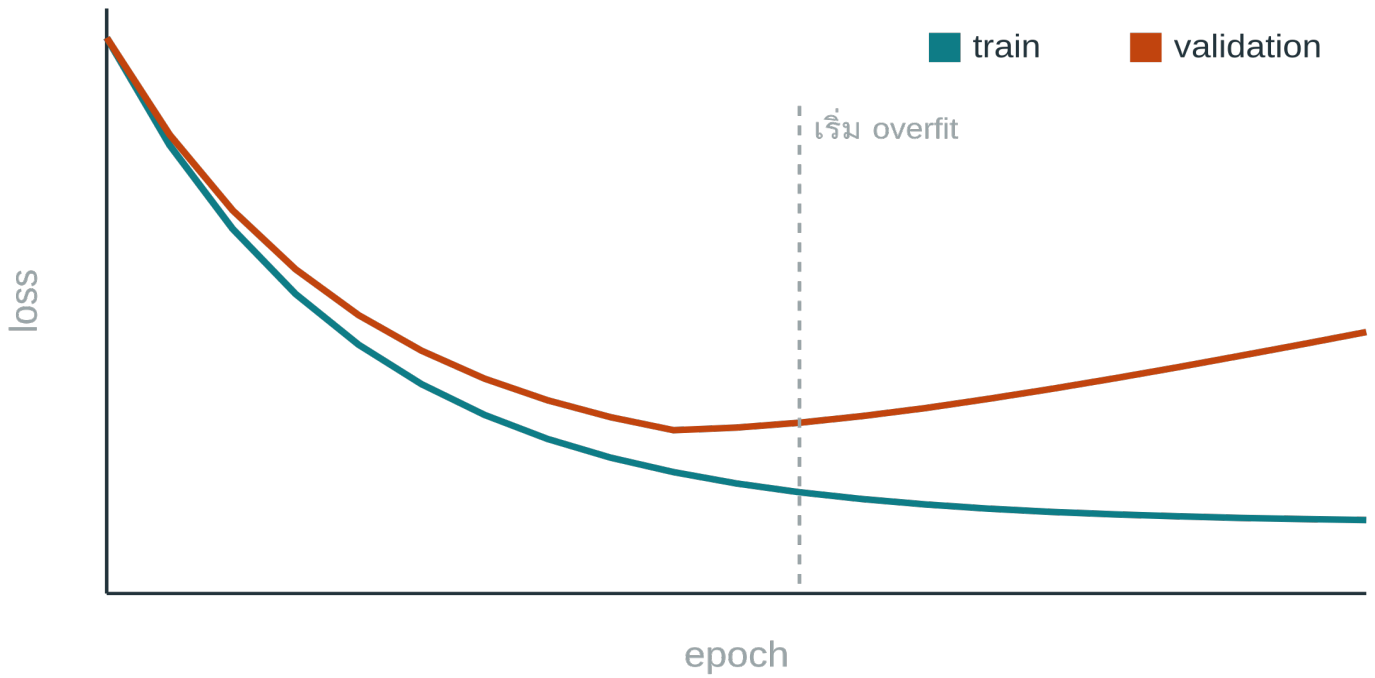
epochs=10 ดูข้อมูลครบทั้งชุด 10 รอบ

batch_size=32 ปรับ weight ต่อ 32 ภาพ

validation_split=0.1 กัน 10% ไว้ตรวจ overfitting

ส่วนที่ 1 · อ่านผล

Loss curve บอกอะไร



อ่านกราฟให้เป็น

- สองเส้น: train กับ validation
- ลงพร้อมกัน = โมเดลเรียนรู้ดี
- validation กลับสูง = เริ่ม overfit
- จุดต่ำสุดของ validation คือจุดที่ควรหยุด

ส่วนที่ 2

CNN บน Fashion-MNIST

แก้โจทย์เดิมด้วยสถาปัตยกรรมที่เหมาะกับภาพ

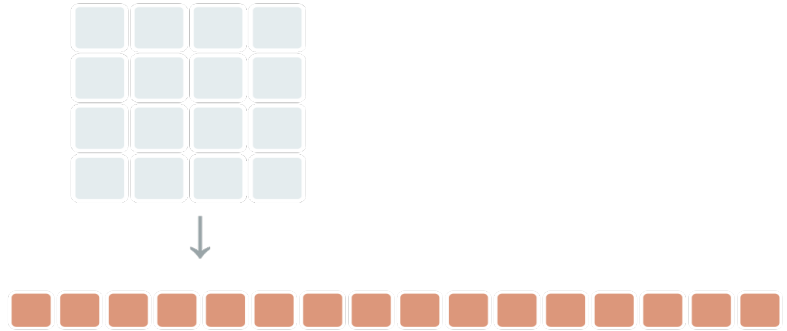
~50 นาที



ส่วนที่ 2 · ทำไมต้อง CNN

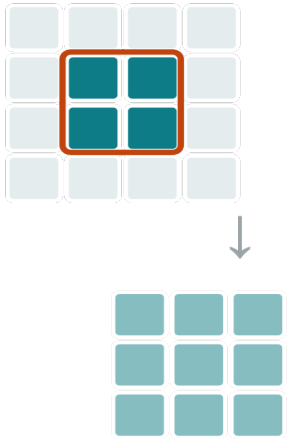
MLP คลี่ภาพทิ้งข้อมูลตำแหน่ง

MLP: คลี่ภาพทิ้งตำแหน่ง



พิกเซลกลายเป็นแถวเดียว ความสัมพันธ์หาย

CNN: รักษาโครงสร้าง 2 มิติ



filter เลื่อนทั่วภาพ เก็บความสัมพันธ์ไว้

MLP มองทุกพิกเซลเป็นค่าอิสระ แต่ CNN รักษาโครงสร้าง 2 มิติไว้ จึงเรียนรู้ภาพได้ดีกว่า

ส่วนที่ 2 · เตรียมข้อมูล

เพิ่มมิติช่องสี

```
x_train_cnn = \
  x_train[..., np.newaxis]
x_test_cnn = \
  x_test[..., np.newaxis]

print(x_train_cnn.shape)
# (60000, 28, 28, 1)
```

CNN ต้องการรูปแบบ (สูง, กว้าง, ช่องสี)

ภาพขาวดำมี 1 ช่องสี ภาพสีมี 3 ช่อง

newaxis เพิ่มมิติจาก (28,28) เป็น (28,28,1)

ลืมขั้นนี้ = error เรื่อง shape ทันที

ส่วนที่ 2 · สร้างโมเดล

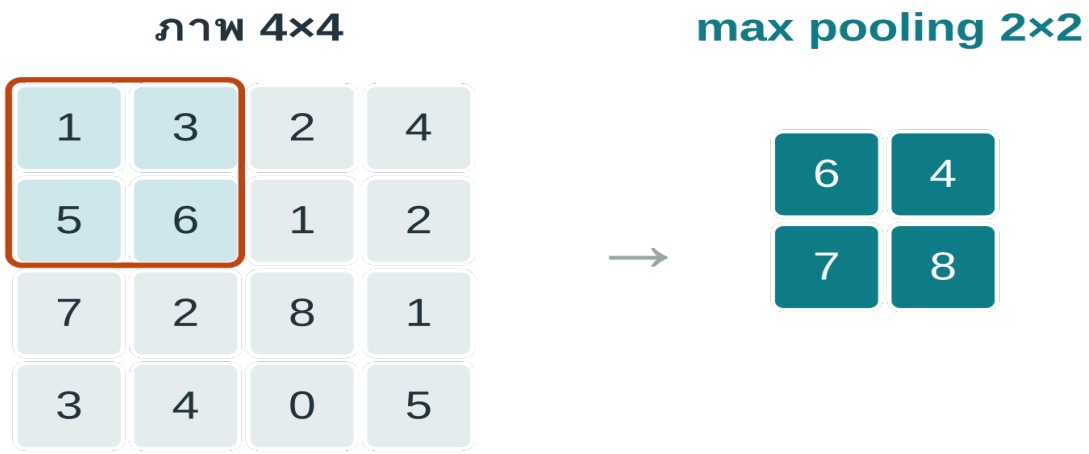
โครงสร้าง CNN มาตรฐาน

```
cnn = keras.Sequential([
  layers.Input((28,28,1)),
  layers.Conv2D(32, 3,
     activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(64, 3,
     activation='relu'),
  layers.MaxPooling2D(),
  layers.Flatten(),
  layers.Dense(64,
     activation='relu'),
  layers.Dense(10,
     activation='softmax')
])
```

- Conv2D(32, 3) = filter 32 ตัว ขนาด 3×3
- MaxPooling2D() ย่อภาพครึ่งหนึ่ง
- ทำซ้ำ Conv→Pool เพิ่ม filter เป็น 64
- Flatten แล้ว Dense เหมือน MLP

ส่วนที่ 2 · Pooling

Max pooling ย่อภาพ เก็บค่าเด่น



เลือกค่าสูงสุดในแต่ละกรอบ ภาพเล็กลงครึ่งหนึ่ง

ชั้น pooling ไม่มี parameter ให้เรียนรู้ เป็นเพียงการเลือกค่าสูงสุดในแต่ละกรอบ

ส่วนที่ 2 · summary

อ่านการหดของภาพทีละชั้น

```
cnn.summary()

# Conv2D_1 -> (26,26,32)
# Pool_1   -> (13,13,32)
# Conv2D_2 -> (11,11,64)
# Pool_2   -> (5,5,64)
# Flatten  -> (1600,)
# Total ~ 121,930
```

28→26 เพราะ conv 3×3 ตัดขอบด้านละ 1

26→13 เพราะ pooling ย่อครึ่ง

11→5 ปัดลง เพราะหารสองไม่ลงตัว

params ส่วนใหญ่พองที่ Flatten→Dense

ส่วนที่ 2 · compile และ fit

เปลี่ยนแค่โครงสร้างโมเดล ขั้น compile และ fit เหมือน MLP เป๊ะ

```
cnn.compile(optimizer='adam',
            loss='sparse_categorical_crossentropy',
            metrics=['accuracy'])

history_cnn = cnn.fit(x_train_cnn, y_train,
                      epochs=10, batch_size=32,
                      validation_split=0.1)
```

workflow สามขั้นใช้ได้กับทุกโมเดล ครั้งนี้เขียน fit เองได้โดยไม่ต้องดูตัวอย่าง

ส่วนที่ 2 · confusion matrix

ดูว่าโมเดลสับสนคู่ไหน

```
from sklearn.metrics \
  import confusion_matrix, \
  ConfusionMatrixDisplay

y_pred = cnn.predict(
  x_test_cnn
  ).argmax(axis=1)

cm = confusion_matrix(
  y_test, y_pred)
```

- predict คืนความน่าจะเป็นของ 10 คลาส
- argmax เลือกคลาสที่น่าจะเป็นสูงสุด
- แถว=คำตอบจริง คอลัมน์=ที่โมเดลทาย
- มักสับสน Shirt / Pullover / Coat

ส่วนที่ 2 · เทียบผล

MLP กับ CNN ต่างกันอย่างไร

ผลที่คาดว่าจะเห็น

- CNN แม่นกว่า MLP ชัดเจน
- ทั้งที่ params พอ ๆ กัน
- เพราะ Conv ใช้ความสัมพันธ์เชิงพื้นที่
- ตัวเลขจริงเติมในตารางสรุปท้ายวัน

เพราะอะไร

- MLP คลี่ภาพทิ้งข้อมูลตำแหน่ง
- CNN รักษาโครงสร้าง 2 มิติไว้
- filter ตัวเดียวใช้ได้ทั้งภาพ
- จึงเรียนรู้ได้ดีด้วย params น้อย

ส่วนที่ 3

Transfer Learning

ใช้โมเดลสำเร็จรูปกับภาพสีจริง

~40 นาที



ส่วนที่ 3 · โหลดข้อมูล

ดาวน์โหลดชุดรูปอาหาร

```
url = ('.../pizza_steak
        _sushi.zip')
path = keras.utils.get_file(
  'pizza_steak_sushi.zip',
  origin=url,
  extract=True)

data_dir = pathlib.Path(
  path).parent / \
  'pizza_steak_sushi'
```

- ต่างจากสองส่วนแรก: เป็นภาพสีจริง
- get_file ดาวน์โหลดแล้วแตกไฟล์อัตโนมัติ
- 3 คลาส คลาสละไม่กี่ร้อยภาพ ถือว่าน้อยมาก
- เน็ตช้า: เตรียมสำเนา zip บน Drive เผื่อ

ส่วนที่ 3 · data pipeline

อ่านภาพจากโฟลเดอร์อัตโนมัติ

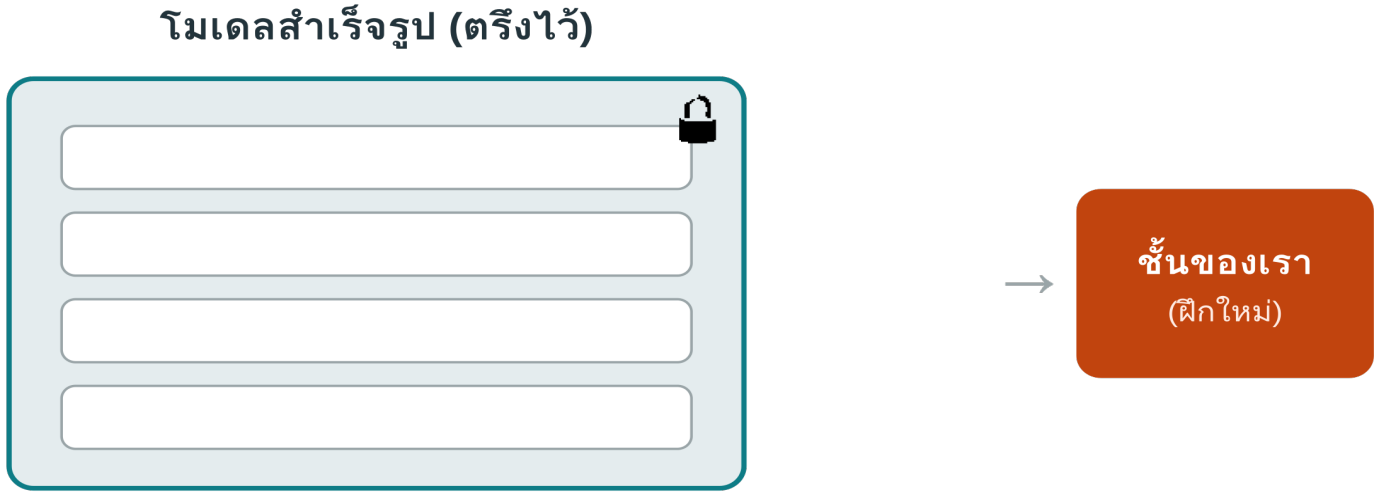
```
train_ds = keras.utils. \
  image_dataset_from_directory(
    data_dir/'train',
    image_size=(224, 224),
    batch_size=32,
    label_mode='int')

class_names = \
  train_ds.class_names
```

- ใช้ชื่อโฟลเดอร์ย่อยเป็นป้ายคลาสอัตโนมัติ
- image_size ปรับทุกภาพเป็น 224×224
- คืนค่าเป็น dataset ที่จัด batch พร้อมฝึก
- API ปัจจุบันของ Keras 3 แทน ImageDataGenerator

ส่วนที่ 3 · แนวคิด

Transfer learning ทำงานอย่างไร



สี่ขั้นตอน

- โหลดโมเดลที่ฝึกกับ ImageNet มาแล้ว
- ตัดชั้นจำแนกเดิมออก (include_top=False)
- ตรึงส่วนที่เหลือไว้ (trainable=False)
- ต่อชั้นจำแนกของเราเอง แล้วฝึกเฉพาะส่วนนั้น

ส่วนที่ 3 · Feature Extraction

สร้างโมเดลจากของสำเร็จรูป

```
base = keras.applications. \
  EfficientNetV2B0(
    include_top=False,
    weights='imagenet',
    input_shape=(224,224,3))
base.trainable = False

model = keras.Sequential([
  base,
  layers.GlobalAverage
        Pooling2D(),
  layers.Dense(3,
    activation='softmax')])
```

include_top=False ตัดชั้นจำแนกเดิมทิ้ง

weights='imagenet' คือความรู้ที่ยืมมา

trainable=False ตรึงไว้ ฝึกเฉพาะที่เราเพิ่ม

Dense(3) เพราะเรามี 3 คลาส

ส่วนที่ 3 · ฝึก feature extraction

ฝึกเร็วเพราะฝึกแค่ชั้นบน

```
model.compile(
  optimizer='adam',
  loss='sparse_
    categorical_crossentropy',
  metrics=['accuracy'])

history_fe = model.fit(
  train_ds,
  validation_data=test_ds,
  epochs=5)
```

- ฝึกแค่ 5 epoch ก็พอ ปรับแค่ Dense ชั้นเดียว
- ความแม่นพุ่งสูงตั้งแต่ epoch แรก ๆ
- ทั้งที่ข้อมูลมีคลาสละไม่กี่ร้อยภาพ
- นี่คือพลังของ transfer learning

ส่วนที่ 3 · Fine-tuning

ปลดล็อกชั้นท้าย ใช้ LR ต่ำมาก

```
base.trainable = True
for layer in \
    base.layers[:-20]:
  layer.trainable = False

model.compile(
  optimizer=keras.optimizers.
    Adam(
    learning_rate=1e-5),
  loss='sparse_
    categorical_crossentropy',
  metrics=['accuracy'])
```

- ปลดล็อกเฉพาะ 20 ชั้นท้าย
- ชั้นต้นที่จับขอบพื้นฐาน ยังตรึงไว้
- learning_rate=1e-5 ต่ำกว่าปกติมาก (ปกติ 1e-3)
- ถ้าปรับแรง จะทำลายความรู้เดิม

ส่วนที่ 3 · สรุป

สรุปผลทั้งวัน — เติมตัวเลขจริงหลังรัน

วิธี	ข้อมูล	accuracy	เวลาฝึก
CNN ฝึกจากศูนย์	Fashion-MNIST เต็ม	___	___
Feature extraction	อาหาร คลาสละไม่กี่ร้อย	___	___
Fine-tuning	อาหาร ชุดเดียวกัน	___	___

ตารางนี้ให้ผู้เรียนกรอกเอง เป็นชิ้นงานส่ง และเอาไปเติมสไลด์เช้าเรื่องข้อมูล 10%

ส่วนที่ 4

โปรเจคของนักศึกษา



เลือกดาต้าเซ็ตใหม่ แล้วลองหลายโมเดลด้วยตนเอง

~40 นาที +

โปรเจค · โจทย์

สิ่งที่ต้องทำ

เลือกดาต้าเซ็ตภาพหนึ่งชุด ที่ไม่ใช่ Fashion-MNIST และไม่ใช่รูปอาหาร

ทำโจทย์จำแนกภาพให้ครบ ตั้งแต่โหลด เตรียม ฝึก จนวัดผล

ทดลองอย่างน้อย 3 แนวทาง แล้วเปรียบเทียบกันในตารางเดียว

อธิบายให้ได้ว่าแนวทางไหนดีที่สุดสำหรับดาต้าเซ็ตของตน และเพราะอะไร

ส่ง notebook ที่รันได้ พร้อมตารางเปรียบเทียบและข้อสรุป

โปรเจค · แนวทาง

ทดลองอย่างน้อย 3 โมเดลบนดาต้าเซ็ตเดียวกัน

แนวทาง A — CNN ที่สร้างเอง ฝึกจากศูนย์ (baseline)

แนวทาง B — Transfer learning แบบ feature extraction (ตรึง base)

แนวทาง C — Transfer learning แบบ fine-tuning (ปลดล็อกชั้นท้าย)

เพิ่มเติม: เปลี่ยนโมเดลสำเร็จรูป MobileNet vs EfficientNet vs ResNet50

เพิ่มเติม: ทดลองผล data augmentation, dropout, จำนวน epoch

โปรเจค · ดาต้าเซ็ต

ดาต้าเซ็ตแนะนำ — ทุกชุดโหลดได้บน Colab

ดาต้าเซ็ต	โหลดด้วย	คลาส	หมายเหตุ
CIFAR-10	`keras.datasets.cifar10`	10	ภาพสี 32×32 — เริ่มง่ายสุด
CIFAR-100	`keras.datasets.cifar100`	100	เหมือน CIFAR-10 แต่ยากขึ้นมาก
Oxford Flowers 102	`tfds 'oxford_flowers102'`	102	ดอกไม้ — เหมาะกับ transfer learning
Beans	`tfds 'beans'`	3	ใบถั่วป่วย/ปกติ — ชุดเล็ก โจทย์จริง
Rock Paper Scissors	`tfds 'rock_paper_scissors'`	3	ภาพมือ — สนุก เห็นผลชัด

แนะนำ: เริ่มด้วย CIFAR-10 (โหลดง่ายสุด) และใช้ Flowers หรือ Beans เพื่อเห็นพลัง transfer learning

โปรเจค · โหลด CIFAR-10

ง่ายที่สุด มากับ Keras เหมือน Fashion-MNIST

```
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()

# ต่างจาก Fashion-MNIST: เป็นภาพสี 3 ช่องอยู่แล้ว
print(x_train.shape)   # (50000, 32, 32, 3)  -> ไม่ต้องเพิ่มมิติช่องสี

x_train = x_train / 255.0   # normalize เหมือนเดิม
```

CIFAR-10 เป็นภาพสีอยู่แล้ว จึงข้ามขั้นเพิ่มมิติช่องสีที่ทำใน Fashion-MNIST ได้เลย

โปรเจค · โหลดผ่าน tfds

สำหรับ Flowers, Beans, Rock-Paper-Scissors

```
import tensorflow_datasets as tfds

(train_ds, test_ds), info = tfds.load(
    'beans',                        # เปลี่ยนชื่อตรงนี้ได้
    split=['train', 'test'],
    as_supervised=True,             # คืนเป็นคู่ (ภาพ, ป้าย)
    with_info=True)

num_classes = info.features['label'].num_classes
```

tfds มีดาต้าเซ็ตพร้อมใช้หลายร้อยชุด เปลี่ยนแค่ชื่อในบรรทัดเดียว

โปรเจค · เตรียมข้อมูลจาก tfds

resize และจัด batch ก่อนฝึก

```
IMG_SIZE = (224, 224)
def prep(image, label):
    image = tf.image.resize(image, IMG_SIZE)
    return image, label

train_ds = train_ds.map(prep).batch(32).prefetch(tf.data.AUTOTUNE)
test_ds  = test_ds.map(prep).batch(32).prefetch(tf.data.AUTOTUNE)
```

จุดที่มักติด: ลืม resize แล้วโมเดลสำเร็จรูป error เรื่อง input shape

โปรเจค · เกณฑ์ส่งงาน

สิ่งที่ต้องมีใน notebook ที่ส่ง

- ระบุดาต้าเซ็ตที่เลือก และเหตุผลสั้น ๆ ที่เลือกชุดนั้น
- โค้ดที่รันได้ครบ ตั้งแต่โหลดจนวัดผล ทั้งสามแนวทาง
- **ตารางเปรียบเทียบ: แนวทาง / accuracy / เวลาฝึก / parameter**
- loss curve ของแต่ละแนวทาง และ confusion matrix ของตัวที่ดีที่สุด
- ข้อสรุป 3-5 บรรทัด: แนวทางไหนดีที่สุด เพราะอะไร เจอปัญหาอะไร

โปรเจค · คำแนะนำ

ข้อควรระวังและเคล็ดลับ

ข้อควรระวัง

- เปิด GPU ก่อนเสมอ ไม่งั้นช้ามาก
- ดาต้าใหญ่: ลองใช้ subset ตอน debug
- อย่าลืม resize ให้ตรง input โมเดล
- fine-tuning ใช้ LR ต่ำเท่านั้น

เคล็ดลับ

- เริ่มจาก baseline ง่าย ๆ ให้รันผ่านก่อน
- เปลี่ยนทีละอย่าง จะได้รู้ว่าอะไรมีผล
- จดผลทุกครั้งที่ทดลอง ลงตารางทันที
- ใช้เวลาฝึกจาก history ประกอบการเทียบ



วันนี้เขียน neural network, CNN และ transfer learning ได้ด้วยตนเองแล้ว

โปรเจค: ลองกับดาต้าเซ็ตของตัวเอง แล้วนำผลมาคุยกันครั้งหน้า

พรุ่งนี้: จากการมองภาพ สู่การเข้าใจภาษา กับ LLM